

# Raport z implementacji interpretera języka DAPL

## Informacje ogólne o projekcie

**Projekt:** Interpreter języka DAPL (Data Analysis Programming Language)

**Przedmiot:** Teoria kompilacji i kompilatory

**Autorzy:** Jan Rolka, Mateusz Sobiech, Maciej Szymański

**Data:** Czerwiec 2025

## 1. Opis języka DAPL

DAPL (Data Analysis Programming Language) to język programowania zaprojektowany specjalnie do analizy danych. Język oferuje wbudowane wsparcie dla struktur tabelarycznych, operacji na danych oraz funkcji analitycznych.

### 1.1 Główne cechy języka:

- Bezkontekstowość:** Język implementuje gramatykę bezkontekstową zgodnie z wymaganiami projektu
- System typów:** Statyczne typowanie z obsługą typów: `int`, `double`, `string`, `bool`, `auto`
- Struktury danych:** Wbudowane wsparcie dla tabel (`Table`) i list (`List`)
- Funkcje:** Pełne wsparcie dla definiowania i wywoływania funkcji z argumentami
- Pętle i instrukcje warunkowe:** `if/elif/else`, `while`, `for`
- Zasięgi zmiennych:** Implementacja lokalnych i globalnych zasięgów z obsługą operatora `^` dla dostępu do zmiennych z wyższych zasięgów
- Operacje na danych:** Filtrowanie, grupowanie, agregacje, wyrażenia lambda

## 2. Architektura interpretera

### 2.1 Ogólna struktura

Interpreter DAPL został zaimplementowany w języku Python z wykorzystaniem narzędzia ANTLR4 do generowania leksera i parsera. Projekt składa się z następujących komponentów:

```
src/
├── DAPL.g4          # Gramatyka ANTLR
├── DAPLLexer.py     # Wygenerowany lexer
├── DAPLParser.py    # Wygenerowany parser
├── DAPLListener.py  # Wygenerowany listener
├── DAPLVisitor.py   # Wygenerowany visitor
├── VariableListner.py # Listener do pierwszego przebiegu
├── FirstVisitor.py  # Visitor do drugiego przebiegu
├── ErrorListener.py # Obsługa błędów syntaktycznych
├── errors.py        # Definicje błędów semantycznych
├── dapl             # Skrypt wykonawczy
├── program.py       # Główna klasa programu
└── main.py          # Punkt wejściowy
```

### 2.2 Wykorzystanie ANTLR

Projekt wykorzystuje ANTLR w wersji 4.13.2 do:

- Automatycznego generowania leksera i parsera na podstawie gramatyki
- Tworzenia drzewa składniowego (AST)
- Implementacji wzorców Visitor i Listener dla przechodzenia po drzewie

## 3. Gramatyka języka DAPL

### 3.1 Pełna gramatyka (DAPL.g4)

Język DAPL został zdefiniowany przez następującą gramatykę bezkontekstową:

```
grammar DAPL;

start: line+ EOF;

line: im
    | dec
    | assign
    | print
```

```

|
| statement
| funcDec
| tableAction
| saveTable
| BlockComment
| LineComment;

im: 'import' NAME ';' #im1
| 'from' NAME 'import' NAME ';' #im2;

statement: scope
| ifStatement
| whileStatement
| forStatement
| functionCall
| returnStatement;

scope: '{' line* '}';

dec:
'var' type ':' NAME ';' #varDecDefaultValue
| 'var' type ':' NAME '=' value ';' #varDecInitValue
| type ':' NAME '=' value ';' #constDec
| 'List' '<' type '>' NAME ';' #emptyListDec
| 'List' '<' type '>' NAME '=' row ';' #listDec
| 'Table' '<' columnList '>' NAME ';' #emptyTableDec
| 'Table' '<' columnList '>' NAME '=' table ';' #tableDec
| 'Table' NAME '=' table ';' #tableDec2
| 'Table' NAME '=' 'read_csv' '(' TEXT ')' ';' #read_csv;

saveTable: NAME '.' 'save' '(' TEXT ')';

columnList: column (',' column)*;

column: type ':' NAME;

table: '[' '[' row ']' (',' '[' row '])* ']';

row: value (',' value)*;

tableAction:
NAME ('.' tableOperation)* ('.' terminalTableOperation)?;

tableOperation: 'filter' '(' INT ',' lambdaExpr ')' #filterOp
| 'apply' '(' INT ',' lambdaExpr ')' #applyOp
| 'addColumn' '(' '<' columnList '>' ')' #addOp
| 'addRow' '(' '[' row ']' ')' #addRow
| 'drop' '(' INT ')' #dropOp
| 'join' '(' NAME ',' INT ',' INT ')' #joinOp
| 'join' '(' NAME ',' NAME ',' NAME ')' #joinOp2
| 'groupBy' '(' INT ',' INT ',' agregation ')' #groupOp;

agregation:
'max'
| 'min'
| 'count'
| 'average'
| 'sum';

terminalTableOperation:
'show' '(' ')' ';' #showOp;

funcDec:
'function' NAME '(' paramList? ')' (':' (type | 'void'))? '{' line* '}';

```

```

paramList:
    param (',' param)*;

param:
    type ':' NAME;

functionCall:
    NAME '(' argList? ')' ';' ;

argList:
    value (',' value)*;

returnStatement:
    'return' (':' value)? ';' ;

//printowanie
print: 'out' '(' value ')' ';' ;

//value - to co printujemy lub przypisujemy
value: expr #valueExpr
    | TEXT #valueText
    | boolExpr #valueBoolExpr
    | 'null' #null
    | '(' type ')' value #cast;

bool: 'true'
    | 'false';

//wyrażenie matematyczne
expr
: expr op=('*' | '/') expr # mulDiv
| expr op=('+' | '-') expr # addSub
| expr '%' expr # modulo
| '(' expr ')' # parens
| '-' expr # minusExpr
| '+' expr # plusExpr
| NAME '(' (value (',' value)*)? ')' # funcCallExpr
| INT # int
| DOUBLE # double
| NAME # nameExpr
| ('^')+ NAME # parent
;

type: 'int'
    | 'double'
    | 'char'
    | 'string'
    | 'bool'
    | 'auto';

//wyrażenia logiczne
boolExpr: orExpr;

orExpr: andExpr (OR andExpr)* ;

andExpr: notExpr (AND notExpr)* ;

notExpr: NOT? primaryBoolExpr;

primaryBoolExpr: bool
    | NAME
    | comparasion
    | NAME '(' (value (',' value)*)? ')'
    | '(' boolExpr ')';

AND : 'and' ;

```

```

AND      : 'and' ;
OR       : 'or' ;
NOT      : '!' ;

ifStatement: 'if' '(' boolExpr ')' '{' line* '}' elifStatement* elseStatement?;

elifStatement: 'elif' '(' boolExpr ')' '{' line* '}' ;

elseStatement: 'else' '{' line* '}' ;

whileStatement: 'while' '(' boolExpr ')' '{' line* '}' ;

comparasion: expr operator expr ;

forStatement:  'for' '(' type ':' NAME '=' expr ';' boolExpr ';' NAME '=' expr ')' '{' line* '}' ;

operator: '=='
| '>'
| '>='
| '<'
| '<='
| '!=' ;

assign: ('^')* NAME '=' value ';' ;

lambdaExpr: NAME '=>' value ;

BlockComment: '/*' .*? '*/' -> channel(HIDDEN);

LineComment: '//' ~[\r\n]* -> channel(HIDDEN);

//Terminale
INT: [0-9]+ ;
DOUBLE: [0-9]+'.'[0-9]+;
NAME: [a-zA-Z][a-zA-Z0-9]*;
TEXT: '"' ~'"'* '"'; // Ciąg znaków w cudzysłowie
WS: [ \t\r\n]+ -> skip ;

```

## 3.2 Omówienie głównych komponentów gramatyki

### 3.2.1 Struktura programu

- **start**: Główna reguła - program składa się z co najmniej jednej linii zakończonej EOF
- **line**: Definiuje wszystkie możliwe konstrukcje na poziomie linii programu

### 3.2.2 Deklaracje i importy

- **im**: Obsługa importów - `import NAME` lub `from NAME import NAME`
- **dec**: Kompletny system deklaracji zmiennych, stałych, list i tabel z różnymi wariantami inicjalizacji

### 3.2.3 System typów

- **type**: Podstawowe typy danych (`int`, `double`, `char`, `string`, `bool`, `auto`)
- **value**: Uniwersalna reprezentacja wartości z obsługą rzutowania typów

### 3.2.4 Wyrażenia arytmetyczne

- **expr**: Hierarchiczna definicja wyrażeń arytmetycznych z poprawnym pierwszeństwem operatorów:
  - Mnożenie i dzielenie (`*`, `/`) - najwyższy priorytet
  - Dodawanie i odejmowanie (`+`, `-`)
  - Modulo (`%`)
  - Wyrażenia nawiasowane
  - Operatory jednoargumentowe (`-`, `+`)
  - Wywołania funkcji w wyrażeniach
  - Dostęp do zmiennych nadrzędnych (`^`)

### 3.2.5 Wyrażenia logiczne

- **boolExpr**: Hierarchiczna struktura wyrażeń logicznych:
  - **orExpr**: Operacje logiczne OR
  - **andExpr**: Operacje logiczne AND (wyższy priorytet niż OR)
  - **notExpr**: Negacja logiczna
  - **primaryBoolExpr**: Podstawowe elementy logiczne (wartości bool, porównania, wywołania funkcji)

### 3.2.6 Instrukcje sterujące

- **ifStatement/elifStatement/elseStatement**: Pełna struktura instrukcji warunkowych z dowolną liczbą `elif`
- **whileStatement**: Pętla while z warunkiem logicznym
- **forStatement**: Pętla for ze zmienną iteracyjną, warunkiem i krokiem
- **scope**: Bloki kodu ograniczone nawiasami klamrowymi

### 3.2.7 Funkcje

- **funcDec**: Deklaracja funkcji z opcjonalnymi parametrami i typem zwracanym
- **functionCall**: Wywołania funkcji z listą argumentów
- **paramList/param**: Definicja parametrów funkcji z typami
- **argList**: Lista argumentów przy wywołaniu
- **returnStatement**: Instrukcja return z opcjonalną wartością

### 3.2.8 Operacje na tabelach

- **tableAction**: Łańcuchowe operacje na tabelach
- **tableOperation**: Zestaw operacji: filter, apply, addColumn, addRow, drop, join, groupBy
- **terminalTableOperation**: Operacje kończące (show)
- **lambdaExpr**: Wyrażenia lambda dla operacji na kolumnach
- **agregation**: Funkcje agregujące (max, min, count, average, sum)

### 3.2.9 Struktury danych

- **table**: Definicja tabeli jako dwuwymiarowej struktury
- **row**: Wiersz jako lista wartości
- **columnList/column**: Definicja kolumn z typami

### 3.2.10 Operatory i porównania

- **operator**: Operatory porównania (`==`, `!=`, `<`, `>`, `<=`, `>=`)
- **comparasion**: Porównania między wyrażeniami

### 3.2.11 Przypisania i dostęp do zmiennych

- **assign**: Przypisania z obsługą operatora `^` dla dostępu do zasięgów nadrzędnych

### 3.2.12 Komentarze i terminale

- **BlockComment/LineComment**: Obsługa komentarzy blokowych `/* */` i liniowych `//`
- **Terminale**: Definicje tokenów (INT, DOUBLE, NAME, TEXT, WS)

## 3.3 Cechy charakterystyczne gramatyki

1. **Bezkontekstowość**: Gramatyka spełnia wymagania gramatyki bezkontekstowej
2. **Hierarchiczność operatorów**: Poprawne pierwszeństwo operatorów arytmetycznych i logicznych
3. **Rekurencyjność**: Obsługa wywołań rekurencyjnych funkcji
4. **Struktury zagnieżdżone**: Bloki kodu, wyrażenia nawiasowane, operacje łańcuchowe
5. **Elastyczność typów**: System typów z automatycznymi konwersjami i rzutowaniem
6. **Zaawansowane struktury danych**: Wbudowane wsparcie dla tabel i operacji analitycznych

## 4. Liczba i cel przebiegów interpretera

### 4.1 Dwuprzebiegowa architektura

Interpreter DAPL został zaprojektowany jako **dwuprzebiegowy** zgodnie z wymaganiami projektu:

#### Przebieg 1: Faza definicji (VariableListener)

- **Cel**: Rejestracja wszystkich deklaracji w globalnej przestrzeni nazw
- **Implementacja**: Wzorzec Listener z ANTLR
- **Przetwarzane elementy**:
  - Deklaracje zmiennych (`var type: name`)
  - Deklaracje stałych (`type: name = value`)
  - Definicje funkcji (`function name( ) {...}`)
  - Schematy tabel (`Table<...> name`)
  - Importy bibliotek (`import str`)

```
# Przykład z VariableListener.py
def enterVarDecInitValue(self, ctx: DAPLParser.VarDecInitValueContext):
    name = ctx.NAME().getText()
    type_ = ctx.type_().getText()
    line = ctx.start.line

    # Sprawdzenie podwójnej deklaracji
    if name in self.variables:
        raise DoubleDeclarationError(name, self.variables[name][1], line, column)

    self.variables[name] = [type_, line, None] # [typ, linia, wartość]
```

## Przebieg 2: Faza wykonania (FirstVisitor)

- **Cel:** Wykonywanie instrukcji i ewaluacja wyrażeń
- **Implementacja:** Wzorzec Visitor z ANTLR
- **Przetwarzane elementy:**
  - Wykonywanie instrukcji
  - Ewaluacja wyrażeń
  - Przypisania do zmiennych
  - Wywołania funkcji
  - Operacje na tablicach

```
# Przykład z FirstVisitor.py
def visitAssign(self, ctx: DAPLParser.AssignContext):
    name = ctx.NAME().getText()
    value = self.visit(ctx.value())

    # Sprawdzenie czy zmienna została zadeklarowana
    var_info, scope = self._find_variable_info(name, 0)
    if not var_info:
        raise UnknownNameException(name, ctx.start.line, column)

    self._set_variable_value(name, value, 0, line, column)
```

## 4.2 Uzasadnienie dwuprzebiegowej architektury

1. **Rozdzielenie odpowiedzialności:** Pierwszy przebieg zajmuje się wyłącznie rejestracją symboli, drugi wykonaniem
2. **Obsługa forward references:** Możliwość wywołania funkcji przed jej definicją w kodzie
3. **Wykrywanie błędów:** Wczesne wykrywanie błędów podwójnej deklaracji
4. **Zgodność z wymaganiami:** Implementacja zgodna z założeniami projektu

# 5. Implementacja tablicy symboli

## 5.1 Struktura tablicy symboli

Tablica symboli została zaimplementowana jako zbiór słowników Pythona:

```
class VariableListener:
    def __init__(self):
        self.variables = {} # name: [type_str, line_num, value]
        self.constants = {} # name: [type_str, line_num, value]
        self.functions = {} # name: FunctionDefinition
        self.tables = {} # name: {colName: type_str}
        self.tables_to_heap = {} # name: heap_key_str
        self.heap = {} # heap_key_str: list_of_rows
```

## 5.2 Zarządzanie zasięgami (scopes)

Implementacja stosu zasięgów umożliwia obsługę lokalnych i globalnych zmiennych:

```

class FirstVisitor:
    def __init__(self, variables, constants, ...):
        self.scope_stack: list[dict] = [self.global_variables.copy()]

    def _push_scope(self):
        """Dodaje nowy zasięg lokalny"""
        self.scope_stack.append({})

    def _pop_scope(self):
        """Usuwa aktualny zasięg lokalny"""
        if len(self.scope_stack) > 1:
            self.scope_stack.pop()

    def _find_variable_info(self, name: str, depth: int):
        """Wyszukuje zmienną w stosie zasięgów"""
        if depth == 0: # Szukaj w całym stosie
            for scope in reversed(self.scope_stack):
                if name in scope:
                    return scope[name], scope
            else: # Szukaj na określonej głębokości (operator ^)
                try:
                    scope = self.scope_stack[-depth - 1]
                    if name in scope:
                        return scope[name], scope
                except IndexError:
                    return None, None
        return None, None

```

## 5.3 Obsługa operatora parent (^)

Język DAPL implementuje unikalny operator `^` do dostępu do zmiennych z wyższych zasięgów:

```

{
    int: a = 1;
    {
        int: a = 5;
        int: x = a;      // x = 5 (lokalny a)
        int: y = ^a;     // y = 1 (parent::a)
    }
}

```

```

def visitParent(self, ctx: DAPLParser.ParentContext):
    counter = ctx.getText().count('^') # Liczba znaków ^
    name = ctx.NAME().getText()

    if counter + 1 > len(self.scope_stack):
        raise UnknownNameException(name, ctx.NAME().getSymbol().line, column)

    # Dostęp do zmiennej na odpowiednim poziomie stosu
    if name in self.scope_stack[-counter-1]:
        return self.scope_stack[-counter-1][name][2] # Zwróć wartość

```

## 6. Implementacja rekordów aktywacji (ramek stosu)

### 6.1 Zarządzanie wywołaniami funkcji

Rekordy aktywacji są implementowane poprzez stos zasięgów i mechanizm wyjątków dla obsługi `return` :

```

def visitFunctionCall(self, ctx: DAPLParser.FunctionCallContext):
    func_name = ctx.NAME().getText()
    func_def = self.functions[func_name]

    # Kontrola głębokości rekurencji
    self._in_function_call_depth += 1
    if self._in_function_call_depth > self.MAX_DAPL_RECURSION_DEPTH:
        raise MaxRecursionDepthExceededError(func_name, call_line, self.MAX_DAPL_RECURSION_DEPTH)

    try:
        # Utworzenie nowego zakresu dla funkcji
        self._push_scope()
        local_scope = self._get_current_scope()

        # Inicjalizacja parametrów w lokalnym zasięgu
        for i, (param_name, param_type_str) in enumerate(func_def.param_names_types):
            arg_val = actual_arg_values[i]
            coerced_arg_val, error = self._coerce_value_to_type(arg_val, param_type_str, ...)
            local_scope[param_name] = [param_type_str, call_line, coerced_arg_val]

        # Wykonanie ciała funkcji
        for line_ctx in func_def.body_ctx_list:
            self.visit(line_ctx)

    except FunctionReturnException as e:
        # Obsługa instrukcji return
        returned_value = e.value
    finally:
        # Czyszczenie stosu
        self._pop_scope()
        self._in_function_call_depth -= 1

```

## 6.2 Struktura rekordu aktywacji

Każdy rekord aktywacji zawiera:

- **Parametry funkcji:** Przechowywane w lokalnym zasięgu
- **Zmienne lokalne:** Deklarowane wewnątrz funkcji
- **Informacje o kontroli:** Głębokość rekurencji, linia wywołania
- **Typ zwracany:** Sprawdzanie zgodności typu zwracanej wartości

```

class FunctionDefinition:
    def __init__(self, name, params_ctx, return_type_repr, body_ctx_list, definition_line):
        self.name = name
        self.param_names_types = [] # Lista (nazwa, typ) parametrów
        self.return_type_str = 'void'
        self.body_ctx_list = body_ctx_list # Lista kontekstów instrukcji
        self.definition_line = definition_line

```

## 6.3 Obsługa rekurencji

Interpreter implementuje zabezpieczenie przed przepełnieniem stosu:

```

MAX_DAPL_RECURSION_DEPTH = 30

def visitFunctionCall(self, ctx):
    if self._in_function_call_depth > self.MAX_DAPL_RECURSION_DEPTH:
        raise MaxRecursionDepthExceededError(func_name, call_line, self.MAX_DAPL_RECURSION_DEPTH)

```

# 7. System typów i konwersje

## 7.1 Podstawowe typy danych

```
def _get_value_type_str(self, value) -> str:
    if isinstance(value, bool): return "bool"
    if isinstance(value, int): return "int"
    if isinstance(value, float): return "double"
    if isinstance(value, str): return "string"
    if value is None: return "null"
    return "unknown"
```

## 7.2 Automatyczne konwersje typów

```
def _coerce_value_to_type(self, value, target_type_str, name_for_error, line_for_error, column_for_error):
    current_value_type_str = self._get_value_type_str(value)

    if target_type_str == current_value_type_str:
        return value, None

    try:
        if target_type_str == "int":
            if isinstance(value, float) and value.is_integer():
                return int(value), None
            return int(value), None
        elif target_type_str == "double":
            if isinstance(value, int):
                return float(value), None
            return float(value), None
        elif target_type_str == "string":
            if isinstance(value, bool):
                return "true" if value else "false", None
            return str(value), None
        # ... inne konwersje
    except (ValueError, TypeError):
        return None, WrongTypeException(name_for_error, target_type_str, line_for_error, column_for_error)
```

## 8. Obsługa błędów i diagnostyka

### 8.1 Hierarchia błędów

Projekt implementuje rozbudowany system błędów podzielony na kategorie:

**Błędy syntaktyczne:**

```
class CustomErrorListener(ErrorListener):
    def syntaxError(self, recognizer, offendingSymbol, line, column, msg, e):
        # Inteligentne analizowanie błędów syntaktycznych
        # Sugestie poprawek
        # Precyzyjne lokalizowanie błędów
```

**Błędy semantyczne:**

- `DoubleDeclarationError` : Podwójna deklaracja symbolu
- `UnknownNameException` : Użycie niezadeklarowanej zmiennej
- `WrongTypeException` : Niezgodność typów
- `FunctionNotFoundException` : Wywołanie nieistniejącej funkcji
- `ArgumentCountException` : Błędna liczba argumentów
- `ReturnTypeErrorMismatchException` : Niezgodność typu zwracanego

### 8.2 Zaawansowana diagnostyka

```
class DoubleDeclarationError(Exception):
    def printError(self, source_lines: list[str]) -> None:
        print(f"[DoubleDeclarationError] Double declaration of '{self.varName}'")
        print(f"First declared at line {self.line1}")
        print(f"Second declaration: line {self.line2}, column {self.column}")
        print(f"{self.line2}: {source_lines[self.line2 - 1].rstrip()}")
        print(" " * (self.column + 2) + "^")
```

## 8.3 Sugestie naprawy błędów

Implementacja inteligentnych sugestii przy błędach syntaktycznych:

```
def syntaxError(self, recognizer, offendingSymbol, line, column, msg, e):
    # Analiza błędu "no viable alternative"
    pattern = r"no viable alternative at input '(.+?)'"
    match = re.search(pattern, msg)
    if match:
        offending_input = match.group(1)
        close = difflib.get_close_matches(offending_input, self.keywords)
        if close:
            error_msg = f'Did you mean "{close[0]}" at line {line}, column {column}?'
            raise SyntaxError(error_msg)
```

## 9. Funkcjonalności specjalne

### 9.1 Operacje na tabelach

DAPL oferuje zaawansowane operacje na strukturach tabelarycznych:

```
Table<int:id, string:name, int:age> users;
users.filter(2, x => x > 18)          // Filtrowanie wierszy
    .groupBy(1, 2, average)          // Grupowanie z agregacją
    .show();                          // Wyświetlenie rezultatu
```

### 9.2 Lambda wyrażenia

```
def visitFillterOp(self, ctx: DAPLParser.FillterOpContext):
    n_column = int(ctx.INT().getText())
    lambda_var = ctx.lambdaExpr().NAME().getText()

    # Utworzenie lokalnego zakresu dla lambda
    self._push_scope()
    self.scope_stack[-1][lambda_var] = [column_type, line, None]

    # Ewaluacja wyrażenia lambda dla każdego wiersza
    for row in self.heap[heap_key]:
        self._perform_assignment(lambda_var, row[n_column], 0, line, column)
        condition = self.visit(ctx.lambdaExpr().value())
        # Filtrowanie na podstawie wyniku
```

### 9.3 Operacje na CSV

```
Table t1 = read_csv("data.csv"); // Automatyczne wczytanie i typowanie
```

## 10. Testowanie języka

### 10.1 Przykłady testowe

Projekt zawiera zestaw przykładów testowych w katalogu `examples/`

## 10.2 Pokrycie funkcjonalności:

- Podstawowe operacje arytmetyczne i logiczne
- Deklaracje zmiennych i stałych
- Instrukcje warunkowe (if/elif/else)
- Pętle (while, for)
- Definicje i wywołania funkcji
- Rekurencja z ograniczeniem głębokości
- Zasięgi zmiennych i operator parent ( ^ )
- Operacje na tabelach (filtrowanie, grupowanie)
- System typów i konwersje
- Obsługa błędów z precyzyjnymi komunikatami

## 11. Zgodność z wymaganiami projektu

---

### 11.1 Wymagania projektu

- ☒ Język bezkontekstowy (ale nie regularny)
- ☒ Zmienne z zasięgami (scopes)
- ☒ Operacje arytmetyczne z nawiasowaniem
- ☒ Typ logiczny i operacje logiczne
- ☒ Porównywanie zmiennych numerycznych
- ☒ Instrukcje warunkowe (if)
- ☒ Pętle/iteracje (for/while)
- ☒ funkcje z argumentami
- ☒ Przekazywanie argumentów przez wartość
- ☒ Rekurencja
- ☒ Komunikaty o błędach z numerem linii
- ☒ Zasięgi obowiązywania zmiennych
- ☒ Dostęp do zmiennych w zasięgu nadrzędnym ( ^a )
- ☒ Automatyczne konwersje typów
- ☒ Zaawansowana diagnostyka błędów
- ☒ Sugestie naprawy błędów
- ☒ Dwuprzebiegowy proces kompilacji

### 11.2 Elementy dodatkowe

- ☒ Struktury tabelaryczne z operacjami analitycznymi
- ☒ wyrażenia lambda
- ☒ Import bibliotek
- ☒ Operacje na plikach CSV
- ☒ Agregacje (sum, average, min, max, count)
- ☒ Operacje join na tabelach
- ☒ System komentarzy (// oraz /\* \*/)

## 12. Podsumowanie techniczne

---

### 12.1 Architektura

- **Parser:** ANTLR4 z gramatyką bezkontekstową
- **Semantyka:** Dwuprzebiegowa analiza (Listener + Visitor)
- **Wykonanie:** Interpreter drzewa składniowego
- **Typy:** Statyczne typowanie z konwersjami
- **Pamięć:** Stos zasięgów + heap dla struktur tabelarycznych

### 12.2 Wydajność

- **Rekurencja:** Ograniczona do 30 poziomów
- **Pamięć:** Efektywne zarządzanie stosem zasięgów
- **Błędy:** Wczesne wykrywanie w fazie analizy

### 12.3 Rozszerzalność

- **Biblioteki:** Modułarna struktura importów
- **Typy:** Łatwe dodawanie nowych typów danych
- **Operacje:** System operacji na tabelach